EC1.408: Digital VLSI Design

International Institute of Information Technology Hyderabad

4-bit Array Multiplier from RTL to GDSII

Open-source flow on the SkyWater SKY130 process

Assignment Report

Github Repository



Name	Roll number
Aryan Shrivastava	2023102025
Kiruba Pandi Selvakumar	2023112010

Icarus Verilog · Yosys · OpenSTA · LibreLane (OpenROAD, Magic, KLayout, Netgen)

Contents

1 Part A - RTL Design and Verification **1**

1.1 Simulation result (PASS line) . . . 1

1.2 Error-detection test (FAIL line) . . . 1

1.3 Hand-worked example . . . 1

1.4 Waveform with the 2-cycle latency . . . 2

1.5 Block diagram . . . 2

2 Part B - Logic Synthesis with Yosys **3**

2.1 Metric Table . . . 3

2.2 Simulation result (PASS line) . . . 3

3 Part C - Static Timing Analysis with OpenSTA **3**

3.1 Result Table . . . 3

3.2 Critical Path List . . . 4

4 Part D - Physical Design: RTL to GDSII **4**

4.1 Results . . . 4

4.2 Post-route maximum frequency . . . 4

4.3 Cells added by the flow . . . 5

4.4 Layout (task 5) . . . 5

5 Section 10: Questions **6**

5.1 Question 1 - Number of logic cells . . . 6

5.2 Question 2 — Critical path . . . 6

5.3 Question 3 — Flip-flop share of the area . . . 7

5.4 Question 4 — Post-route $F_{\max}$ and the cells added by the flow . . . 7

5.5 Question 5 — DRC and LVS . . . 7

5.6 Question 6 — Scaling to an 8-bit multiplier . . . 7

6 Contributions **8**

Contributions **8**

1 Part A - RTL Design and Verification

1.1 Simulation result (PASS line)

```
(nix:devshell-env) [nix-shell:~/Documents/DVD/mult_array]$ iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:95: $finish called at 7700000 (1ps)
```

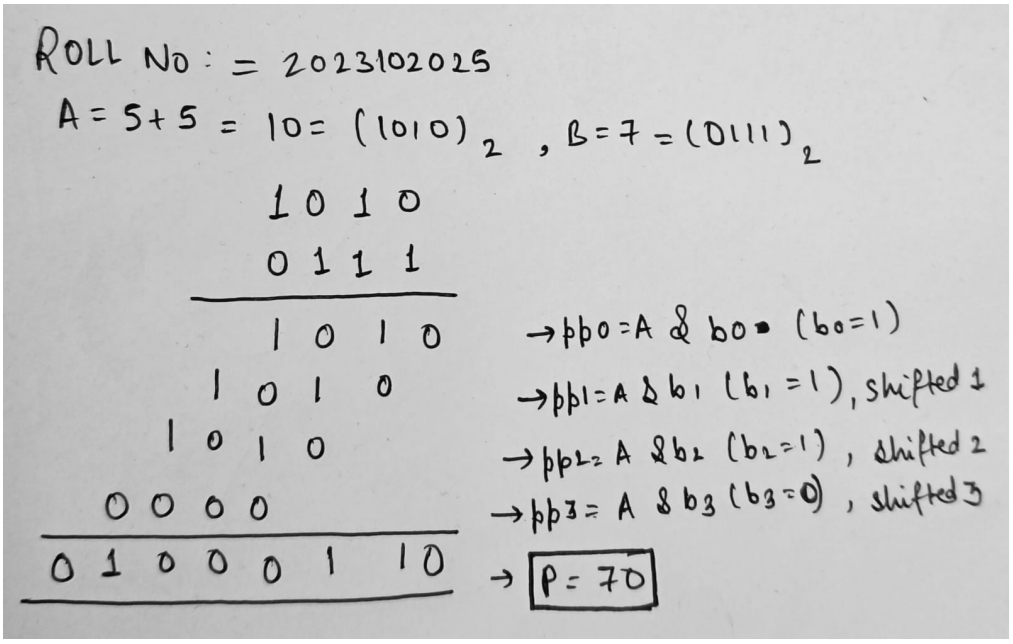
1.2 Error-detection test (FAIL line)

```
sed -i 's/assign sum  = a ^ b ^ cin;/assign sum = a ^ b;/' rtl/full_adder.v
grep "assign sum" rtl/full_adder.v                          # assign sum = a ^ b;
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim | head -3
vvp sim | grep -E "^(PASS|FAIL)"

Output:
MISMATCH: a=3 b=3 expected p=9 got p=5 (t=1580 ns)
MISMATCH: a=3 b=6 expected p=18 got p=10 (t=1670 ns)
FAIL: 256 vectors checked, 95 errors
```

```
    assign sum  = a ^ b;
VCD info: dumpfile mult.vcd opened for output.
MISMATCH: a=3 b=3 expected p=9 got p=5 (t=1580 ns)
MISMATCH: a=3 b=6 expected p=18 got p=10 (t=1670 ns)
FAIL: 256 vectors checked, 95 errors
```

1.3 Hand-worked example



1.4 Waveform with the 2-cycle latency

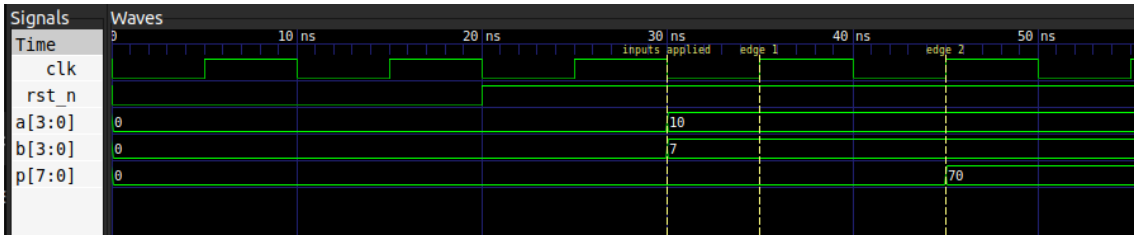


Figure 1: Waveform for Multiplication for 10 and 7

1.5 Block diagram

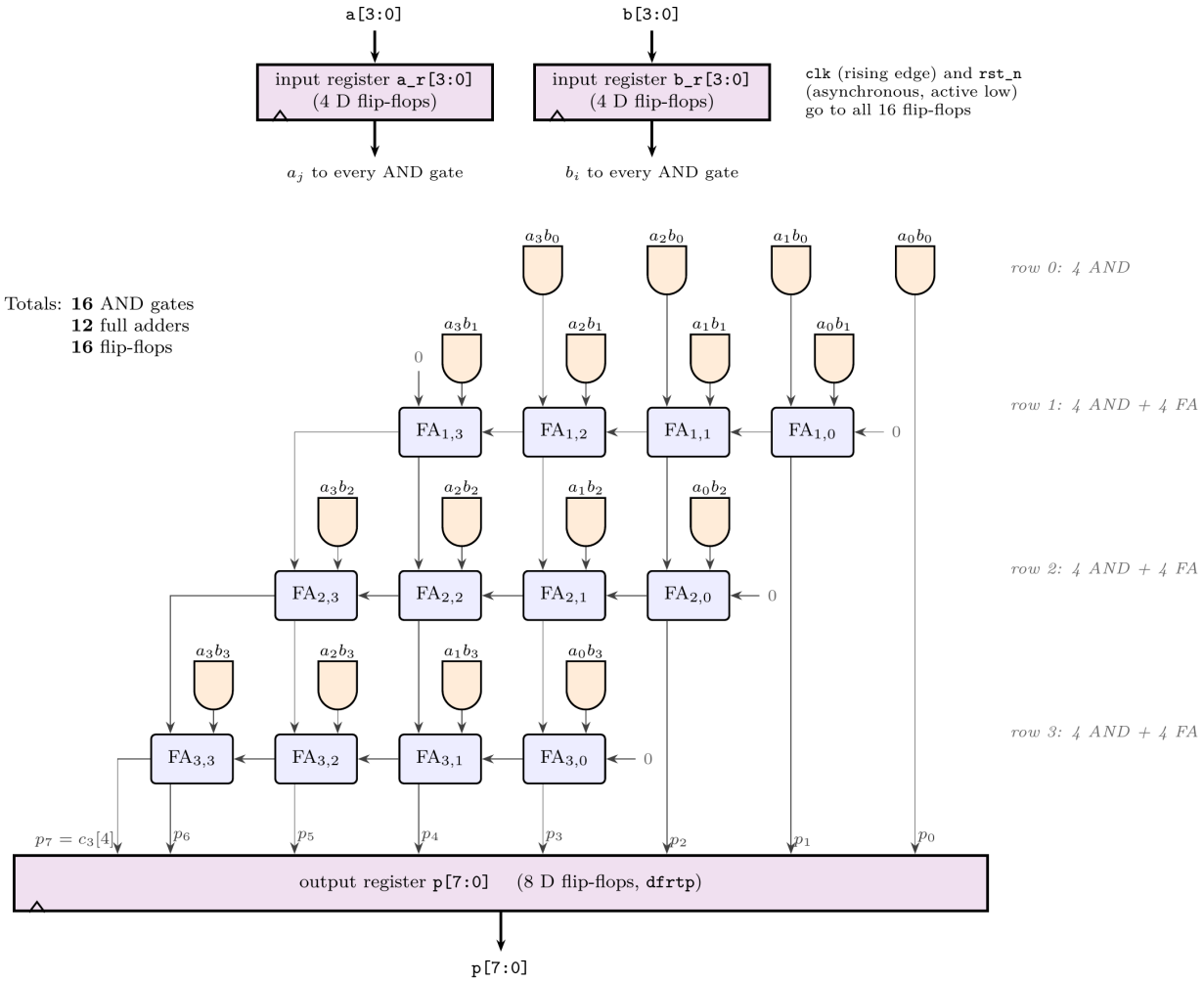


Figure 2: Block Diagram

2 Part B - Logic Synthesis with Yosys

2.1 Metric Table

Metric	Value
Total number of cells	70
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (μm^2)	797.014
Flip-flop area (μm^2)	400.384 (16×25.024)
Flip-flop area (% of total)	50.24 %
The 3 most-used cell types and their counts	`dfrtp_1` × **16**, `nand2_1` × **10**, `clkinv_1` × **6**
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 1: Part B results, from `synth/mult_array_stat.rpt` and the gate-level simulation of `synth/mult_array_netlist.v`.

2.2 Simulation result (PASS line)

```
(nix:devshell-env) [nix-shell:~/Documents/DVD/mult_array]$ \
> CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
$B/iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
  tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
$B/vvp gls
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:95: $finish called at 7700000 (1ps)
```

Figure 3: Pass result

3 Part C - Static Timing Analysis with OpenSTA

3.1 Result Table

Metric	Value
Worst slack at $T = 10$ ns (ns)	7.017
$T_{\min}$ (ns)	$10 - 7.017 = 2.983$ (≈ 2.98)
$F_{\max}$ (MHz)	$1000/2.983 = 335.2$
Slack at $T = T_{\min} = 2.98$ ns (ns)	-0.003 (≈ 0)
Critical path start point	`_109_` (`dfrtp_1`, input register `a_r[1]`)
Critical path end point	`_107_` (`dfrtp_1`, output register `p[7]`)
Number of cells on the critical path	9 (launch flip-flop + 8 logic gates); 10 with the capture flip-flop
Latency (ns) $= 2 \times T_{\min}$	$2 \times 2.983 = 5.97$
Throughput (million results per second)	335.2

Table 2: Part C results (OpenSTA, typical corner `tt_025C_1v80`).

3.2 Critical Path List

#	Instance	Cell	Delay (ns)	Arrival (ns)	Signal in the RTL
1	_109_	dfrtp_1 (CLK→Q)	0.396	0.396	a_r[1] (input register)
2	_054_	a22o_1	0.248	0.644	row 1, $FA_{1,1}$ + partial-product ANDs
3	_055_	and3_1	0.181	0.824	row 1, $FA_{1,1}$
4	_058_	nor3_1	0.276	1.101	$c_2[1]$, carry of $FA_{2,0}$
5	_070_	xnor3_1	0.453	1.554	$s_2[1]$, sum of $FA_{2,1}$
6	_071_	and3_1	0.209	1.763	$c_3[1]$, carry of $FA_{3,0}$
7	_085_	maj3_1	0.361	2.124	$c_3[2]$, carry of $FA_{3,1}$
8	_097_	maj3_1	0.379	2.503	$c_3[3]$, carry of $FA_{3,2}$
9	_098_	maj3_1	0.349	2.852	$c_3[4]$, carry of $FA_{3,3}$
–	_107_	dfrtp_1 (setup 0.131)	–	2.852	p[7] (output register)

Table 3: Cells on the critical path (sta/mult_array_timing.rpt, path group clk). All 8 logic gates come from full-adder logic; the path passes through 7 full adders, close to the estimate of $2N = 8$ full-adder delays.

4 Part D - Physical Design: RTL to GDSII

4.1 Results

Metric	Value
CLOCK_PERIOD used (ns)	4.0
Number of runs needed	1
Core area (μm^2)	17759.5
Cells after synthesis (Part B)	70
Cells after place and route	4795
Kinds of cells added	decap, fill, tap, buffers
Wirelength (µm)	2610
Setup slack (ns)	+1.167 (typical corner; −1.254 at slow corner)
Post-route F_{max} (MHz)	353.0
F_{max} from Part C (MHz)	335.2
DRC errors	0
LVS errors	0

Table 4: Part D results, from pnr/metrics.json.

Which setup slack is used.
pnr/metrics.json reports two setup slacks. The value in Table 4 is taken at the typical corner (timing__setup__ws__corner:nom_tt_025C_1v80 = 1.167 ns), the same corner as the Part C analysis, so that the post-route F_{max} can be compared directly with Part C. At this corner the first run already meets timing, so one run was sufficient.

LibreLane also checks the slow corner ss_100C_1v60 (slow transistors, 100 °C, 1.60 V). There the worst slack is −1.254 ns, which is the overall timing__setup__ws in metrics.json. The flow reports it as a warning only and still completes with 0 DRC and 0 LVS errors. Meeting timing at this corner as well would need CLOCK_PERIOD $\geq 4.0 + 1.254 = 5.254$ ns, i.e. 5.5 ns on the 0.5 ns grid, corresponding to an F_{max} of about 190 MHz (see Question 4).

4.2 Post-route maximum frequency

$$F_{\max}^{\text{post-route}} = \frac{1000}{\texttt{CLOCK_PERIOD} - \text{setup slack}} = \frac{1000}{4.0 - 1.167} = 353.0\,\text{MHz}$$

$$F_{\max}^{\text{Part C}} = 335.2\,\text{MHz}.$$

The post-route $F_{\max}$ is about 18 MHz *higher* than the Part C estimate. Wire delay, the real clock tree and the signoff margins slow the design down, but LibreLane re-synthesizes the RTL with stronger cells and its resizer adds repair buffers on the critical nets, which more than compensates.

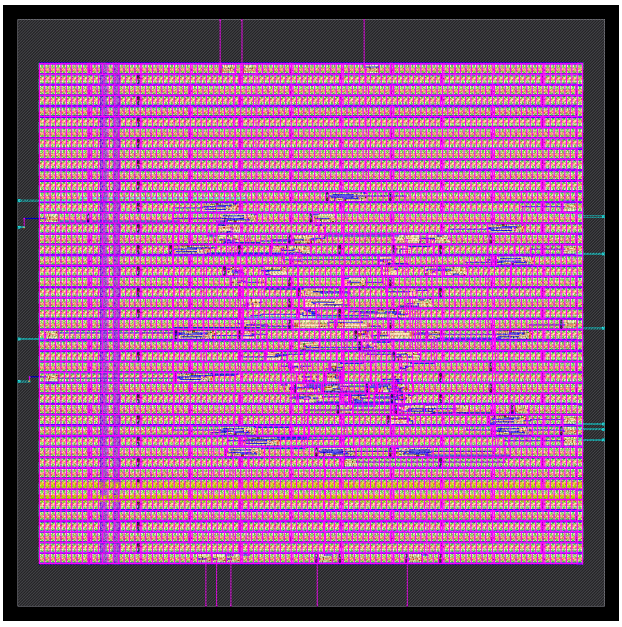
4.3 Cells added by the flow

Kind	Cells	Count
Decap	`decap_3`	4170
Fill	`fill_1` (210), `fill_2` (66)	276
Tap	`tapvpwrvgnd_1`	245
Buffers	`clkdlybuf4s25_1` (19, inputs/outputs/reset fanout)	
	`buf_2` (12, timing repair)	
	`clkbuf_16` (3, clock tree)	
	`dlygate4sd3_1` (2, hold fix)	36
Logic gates	52 gates	52
Flip-flops	`dfrtp_2`	16
Total		**4795**

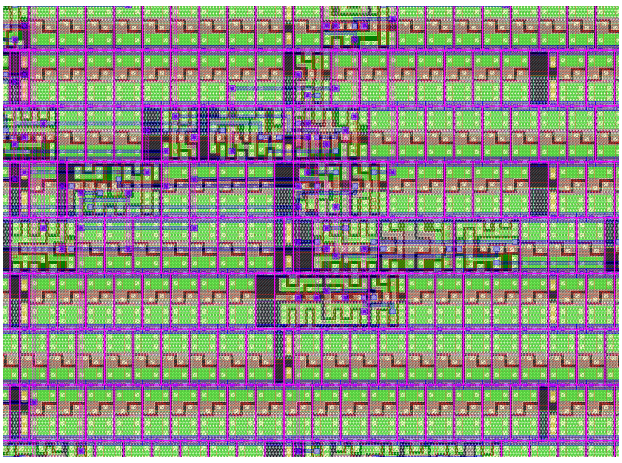
Table 5: Cells after place and route, against 70 after synthesis.

Only 104 of the 4795 cells have a logic function (52 logic gates, 16 flip-flops and 36 buffers); the decap, fill and tap cells are physical cells needed for power integrity, manufacturability and latch-up protection. The purpose of each kind is explained in Question 4.

4.4 Layout (task 5)



(a) Whole die (150 × 150 µm).



(b) Zoomed view (x = 82–112, y = 40–62 µm): standard-cell rows between the VPWR/VGND rails and the metal wires connecting the cells.

Figure 4: Final layout `pnr/mult_array.gds` in KLayout.

5 Section 10: Questions

5.1 Question 1 - Number of logic cells

Answer. Synthesis produced 70 cells, of which 16 are flip-flops, so the logic uses $70 - 16 = \mathbf{54}$ cells rather than $16 + 12 = 28$. The count differs because synthesis does not map each AND gate or full adder to one cell.

First, ABC maps only to single-output cells, so it cannot use SKY130's two-output full-adder cell (`fa`); each full adder becomes an XOR3 for the sum and a MAJ3 for the carry, so the 12 full adders alone need about 24 cells.

Second, `synth -flatten` removes the `full_adder` boundaries and `abc` rebuilds the array as one block: constant propagation ($c_i[0] = 0$, $r_0[3] = 0$) reduces 4 adders to half adders, and the 16 partial-product ANDs merge into AND–OR cells such as `a22o`, `a21oi` and `o21ai`, leaving only 5 `maj3` and 4 `xor3`/`xnor3` cells as intact full-adder pieces.

Third, static CMOS gates are inverting, so ABC builds most functions from NAND, NOR, AOI and OAI cells (26 in total) and adds 6 `clkinv` inverters to correct the polarity.

5.2 Question 2 — Critical path

Supporting results.

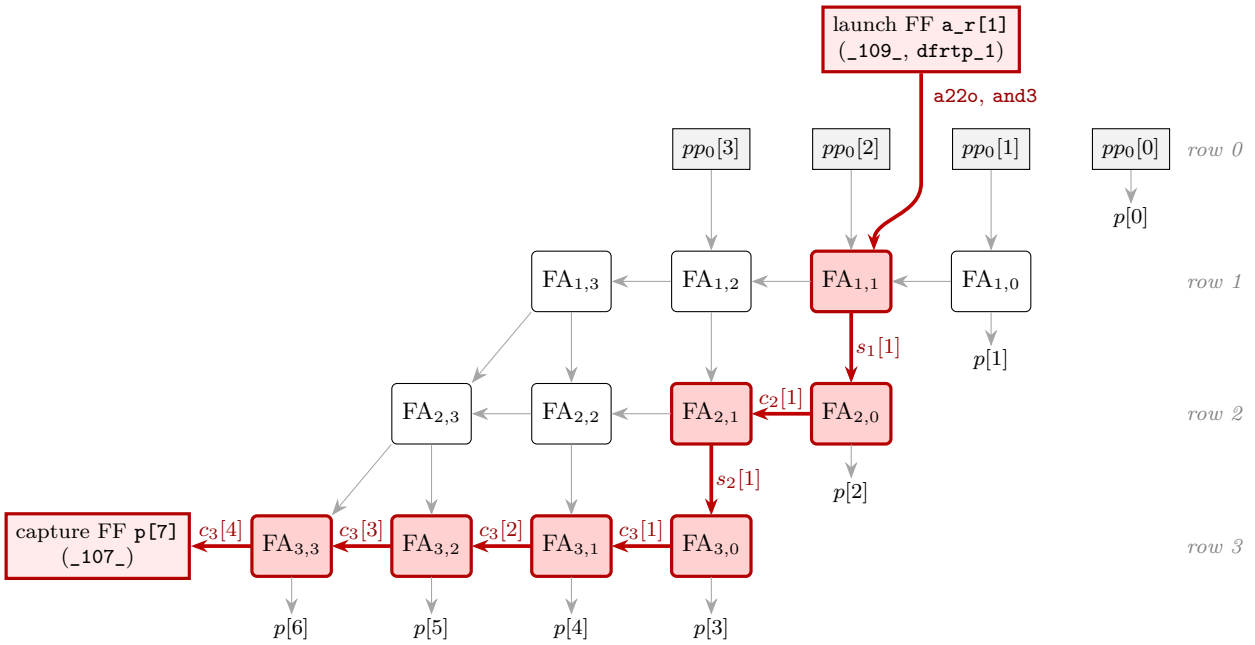


Figure 5: The array, with columns aligned by bit weight (carries move left, sums move down). The critical path reported by OpenSTA is in red: 7 of the 12 full adders, from `a_r[1]` to `p[7]`.

Answer. The critical path runs from input register `a_r[1]` (`_109_`) to output register `p[7]` (`_107_`), whose D input is $c_3[4]$, the final carry of row 3. Its arrival path has **9 cells**, the launching flip-flop and **8 logic gates** (`a22o`, `and3`, `nor3`, `xnor3`, `and3`, `maj3`, `maj3`, `maj3`), or 10 including the capturing flip-flop. Mapped back to the RTL, these gates pass through **7 full adders** ($\mathrm{FA}_{1,1} \to \mathrm{FA}_{2,0} \to \mathrm{FA}_{2,1} \to \mathrm{FA}_{3,0} \to \mathrm{FA}_{3,1} \to \mathrm{FA}_{3,2} \to \mathrm{FA}_{3,3}$): the path zig-zags down the rows through the sum outputs, then ripples through all four carries of row 3. This agrees with the estimate of about $2N = 8$ full-adder delays; the eighth stage, $\mathrm{FA}_{1,0}$, is only a half adder (constant-0 carry-in), and its carry enters the path as a side input rather than as a separate stage.

5.3 Question 3 — Flip-flop share of the area

Answer. The 16 flip-flops occupy $16 \times 25.024\,\mu m^2 = 400.384\,\mu m^2$ of the $797.014\,\mu m^2$ total cell area, i.e. **50.24 %**. The share is large because the design is small: the flip-flop count grows only linearly with the width ($4N = 16$, for 8 input and 8 output bits), while the array logic grows as N^2 (N^2 ANDs and $N(N-1)$ adders), so at $N = 4$ the 54 logic cells ($396.630\,\mu m^2$) take no more area than the registers. Each flip-flop is also large: one `dfrtp_1` ($25.0\,\mu m^2$) equals about 3.4 average logic cells ($7.3\,\mu m^2$), because it is a master–slave latch pair with clock buffering and an asynchronous reset (28 transistors, against 4–14 for a NAND, AOI, XOR2 or MAJ3 gate).

5.4 Question 4 — Post-route F_{max} and the cells added by the flow

Answer. At the typical corner our post-route $F_{max} = 1000/(4.0 - 1.167) = 353.0\,MHz$ is not lower but 18 MHz higher than the 335.2 MHz from Part C. Physical design does add delay: the extracted wire resistance and capacitance (2610 µm of wire), the real clock tree (0.36 ns insertion delay plus skew) and the signoff margins (0.25 ns clock uncertainty, ±5 % derate) together cost 0.46 ns. However, LibreLane re-synthesizes the RTL with stronger `_2` cells, and its resizer upsizes gates and inserts 12 buffers on the critical nets; this netlist is 0.61 ns faster than the Part B netlist, which more than offsets the penalties. At the slow corner (`ss_100C_1v60`) the slack is −1.254 ns, so there F_{max} falls to about 190 MHz. The flow grew the design from 70 to 4795 cells by adding **decap** cells (4170), capacitors between VPWR and VGND that supply switching current locally and reduce IR drop; **fill** cells (276), which keep the n-well, implants and power rails continuous as DRC and density rules require; **tap** cells (245), which tie the n-well to VPWR and the substrate to VGND to prevent latch-up, since the standard cells have no taps of their own; **clock-tree buffers** (3), so one clock pin drives all 16 flip-flops with low skew; **I/O buffers** (19), which isolate the pins from the internal logic and give a known drive and load; **timing-repair buffers** (12), which split heavily loaded nets on the critical path; and **hold buffers** (2), which delay two very short paths so they cannot violate hold time.

5.5 Question 5 — DRC and LVS

Answer. DRC (design rule check) verifies the layout *geometry* against the foundry's manufacturing rules for every mask layer: minimum width and spacing, enclosure of contacts and vias, density and antenna rules.

LVS (layout versus schematic) verifies *connectivity*: the netlist extracted from the layout must match the post-route Verilog netlist cell for cell, pin for pin and net for net. Both must pass because they catch different failures: a DRC-clean layout can still implement the wrong circuit, since a missing via or a legal-looking wire that shorts two nets breaks no geometric rule, and an LVS-clean layout can still be unmanufacturable if correct connections are drawn with shapes that are too narrow or too close. A mask set is expensive and an error in silicon cannot be fixed, so a foundry accepts a GDS only when both checks are clean. Our final layout has **0 DRC errors** (Magic and KLayout) and **0 LVS errors** (Netgen: "Circuits match uniquely").

5.6 Question 6 — Scaling to an 8-bit multiplier

Answer. From Section 2, an N-bit array uses $N(N-1)$ full adders and has a critical path of about $2N$ full-adder delays. For $N = 8$ this gives $8 \times 7 = \mathbf{56}$ full adders instead of $4 \times 3 = 12$ (4.7×), and about $2 \times 8 = \mathbf{16}$ full-adder delays instead of 8, so the critical path doubles. With our Part C numbers, a fixed flip-flop overhead of 0.527 ns (clock-to-Q 0.396 ns + setup 0.131 ns) and 0.307 ns per full-adder delay (the 2.456 ns logic delay divided by 8), the estimate is $T_{min}(8) \approx 0.527 + 16 \times 0.307 = 5.44\,ns$ ($F_{max} \approx 184\,MHz$), against 2.983 ns (335 MHz) for $N = 4$. The period grows by 1.8× rather than 2× because the flip-flop overhead does not scale with N.

6 Contributions

Part	Work	Done by
Part A	RTL design and verification	Kiruba Pandi Selvakumar (2023112010)
Part B	Logic synthesis with Yosys	Aryan Shrivastava (2023102025)
Part C	Static timing analysis with OpenSTA	Kiruba Pandi Selvakumar (2023112010)
Part D	Physical design with LibreLane	Aryan Shrivastava (2023102025)
Report	Results, figures and answers to the questions	Both members

Table 6: Contribution of each team member.